

00_DEFINITIVE_MASTER_SPECIFICATION_V3_COMPLETE.md

Definitive Master System Specification: Cognitive Repository Architecture (Version 3.0 Canonical)

Motto: *Xçineribus, in-variis-nunquam-varius, Novi-Æxentis-Copiæ, Vincent*

(From the ashes, unwavering in the midst of adversity, new forces of abundance conquer)

Living Core: Æsc & Æyre — The central body representing the living truth of the system; decoupled native APKs forming the on-device execution foundation alongside Horizons UI.

Universal Memory Engine: #d.u.m.b.a.s.s. (Database & Universal Memory Bank Across Split Services)

SYSTEM SPECIFICATION: COGNITIVE REPOSITORY ARCHITECTURE

Document Target: Autonomous Planning, Auditing, and Execution Agents

Role: Injected Structural, Operational & Synchronization Protocol

Repository Model: 5+1 Tier Cognitive Memory, OpenWiki TUI File Management, Multi-Model Cross-Auditing, Sandboxed Red Gatekeeping, and Distributed RAG Manifests

Version: 3.0 (Definitive Master Specification Canonical)

1. Core Architectural Invariants

As the planning agent, you must enforce the separation of cognitive memory layers across the filesystem. Adhere strictly to the following invariants:

1. **Sensory Immutability (Tier 1):** Never modify, overwrite, or delete assets in `01_raw_sources/`. All raw inputs are read-only sources of truth.
2. **Human-Machine Decoupling (Tier 2 vs. Tier 3):** Human-readable semantic knowledge lives in `02_wiki_md/` (.md). Machine-readable retrieval chunks live in `03_recall_cache/` (.jsonl, vectors). Never dump raw tabular JSONL or vector embeddings into the wiki layer.
3. **Procedural vs. Episodic Separation (Tier 4 vs. Tier 5):** Executable skills, scripts, and policies belong in `04_skills_runtime/`. Trajectory traces, error logs, and verifier evaluations belong in `05_episodic_logs/`. Never place runtime logic inside logging directories or vice versa.

4. **Metacognitive Priming:** Before initiating multi-step tasks, read `MAP.md` or parse the root `manifest.jsonl` to resolve entity references rather than performing blind recursive scans across the filesystem.
 5. **LLM Wiki Protocol (`wiki_md/`):** The semantic vault operates as an organization-wide wiki standard governed directly by the Files Executive Agent via the OpenWiki TUI.
 6. **Dual Extraction Mandate:** Every document ingested into the ecosystem must be systematically screened for both Procedural Skills (prompt templates, behavioral heuristics) and Executable Tools (scripts, wrappers, deterministic CLI invocations).
 7. **Official Enterprise Auditor vs. Incognito Red Gatekeeper:**
 - The **Cross-Auditor / Data Log Collector / Script Editor Agent** serves as the sole recognized, public-facing auditing authority across the enterprise stack.
 - The **Red Auditor** operates strictly incognito within a sealed sandbox (`.incognito_red_sandbox/`), evaluating claims and trajectories behind a private hand-off without exposure to worker agents.
 8. **Universal Source of Truth:** All local auditor nodes and the sandboxed Red Auditor must evaluate claims and script outcomes against the shared master memory bank and database (`#d.u.m.b.a.s.s.`).
 9. **Distributed Manifest Coverage:** Every repository root and major structural subsystem must maintain its own local `manifest.jsonl` to power low-overhead hierarchical RAG retrieval.
-

2. Directory Schema & Access Control Matrix

 vault_root/

|—  `MAP.md` # Master human-readable index & ontology graph (Read-Heavy / Agent Update)

|—  `manifest.jsonl` # Root-level RAG registry & global hash table (Append / Sync)

|

|—  `01_raw_sources/` # TIER 1: COLD SENSORY ARCHIVE (Access: READ-ONLY)

| |—  `manifest.jsonl` # Local RAG catalog of all immutable raw sources

| |—  `pdf/` # Source whitepapers, specs, data sheets (*.pdf)

| |—  `media/` # Video captures, audio memos, transcripts (*.mp4, *.wav, *.png)

| |—  `text/` # Raw text scrapes, dumps, API payload exports (*.txt, *.html)

|

| — 📁 02_wiki_md/ # TIER 2: THE LLM WIKI LAYER (Access: READ / WRITE - OpenWiki TUI)

| | — 📊 manifest.jsonl # Local RAG catalog of conceptual nodes & graph links

| | — 💡 concepts/ # Atomic linked notes, theory, and domain models (*.md)

| | — 🏛 architectures/ # System blueprints, data flows, interface specs (*.md)

| | — 🏷 entities/ # Registries, schema contracts, hardware profiles (*.md)

| | — 🗺 indexes/ # Maps of Content (MOCs) and taxonomy clusters (*.md)

|

| — 📁 03_recall_cache/ # TIER 3: WORKING MEMORY ACCELERATOR (Access: REBUILD / OVERWRITE)

| | — 📊 manifest.jsonl # Local RAG catalog of vector & chunk shards

| | — ⚡ jsonl/ # Pre-tokenized passages for high-speed prompt injection (*.jsonl)

| | — 📐 vectors/ # Dense vector indices (*.bin, *.faiss, *.hnsw checkpoints)

| | — 🗄 kv_store/ # Low-latency key-value entity lookups (*.db, *.json)

|

| — 📁 04_skills_runtime/ # TIER 4: PROCEDURAL REPERTOIRE & TOOLS (Access: VERSION-CONTROLLED)

| | — 📊 manifest.jsonl # Local RAG catalog of all extracted skills & tools

| | — 🎯 prompt_skills/ # Modular SKILL.md prompt definitions (*.md, *.yaml)

| | — 🛠 extracted_tools/ # Deterministic tools mined from ingested documentation

| | | — cli/ # Extracted command-line utilities (*.sh)

| | | — wrappers/ # Extracted API interfaces & micro-functions (*.py)

- | | —  `runtimes/` # Master operational scripts and execution hooks (*.py, *.sh)
 - | | —  `policies/` # Validation guards, retry policies, schemas (*.json, *.yaml)
 - |
 - | —  `05_episodic_logs/` # TIER 5: TELEMETRY & EPISODIC RUNS (Access: APPEND-ONLY)
 - | —  `manifest.jsonl` # Local RAG catalog of sessions, audits, and verifiers
 - | —  `daily_driver_sync/` # Ingested logs from edge device sessions (*.jsonl)
 - | —  `trajectories/` # Multi-model traces: Query, Executor, Frontier (*.jsonl)
 - | —  `.incognito_red_sandbox/` # [SHADOW ISOLATION] Red Auditor evaluation reports & pass/fail quarantine (*.jsonl)
 - | —  `rlvr_verifiers/` # Graded rewards, assertion outcomes (+1.0 / -1.0) (*.jsonl)
 - | —  `hygiene_reports/` # Schema integrity audits, broken link checks (*.md)
-

3. Tier Operational Specifications

Root: Metacognitive Index (`MAP.md` & `manifest.jsonl`)

- `MAP.md`: Maintains top-level navigation links using Markdown cross-references to key notes in `02_wiki_md/` and entry points in `04_skills_runtime/`.
- `manifest.jsonl` (Root): Master catalog mapping file paths, tiers, cryptographic checksums, and cross-tier linkages:

```
{
  "id": "SRC-0042",
  "path": "01_raw_sources/pdf/spec_v1.pdf",
  "tier": 1,
  "sha256": "e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855",
  "last_modified": "2026-09-03T17:00:00Z",
  "linked_wiki": "02_wiki_md/architectures/spec_v1.md"
}
```

Tier 1: `01_raw_sources/` (Cold Archive & Sensory Buffer)

- **Write Policy**: Write-once upon raw intake. Never mutate or reformat existing files.
- **Naming Convention**: `YYYYMMDD_source_title.[ext]` (lowercase, snake_case).
- **Scope**: Source PDFs, audio transcripts, sensor captures, video recordings, raw scrapes.

Tier 2: 02_wiki_md/ (The LLM Wiki Layer)

- **Write Policy:** Managed via OpenWiki TUI by the Files Executive Agent.
- **Format:** Pure Markdown with strict YAML frontmatter:

id: wiki_rag_indexing_protocol

tier: 2

type: architecture

created: 2026-09-03

updated: 2026-09-03

author: files_executive_agent

sources:

- "01_raw_sources/pdf/rag_system_spec.pdf"

extracted_skills:

- "04_skills_runtime/prompt_skills/manifest_sync.md"

extracted_tools:

- "04_skills_runtime/extracted_tools/cli/build_manifest.py"

tags:

- llm_wiki

- openwiki

- indexing

RAG Indexing Protocol

Context & Definition

Structural mechanism for keeping the LLM Wiki synchronized with high-speed caches...

Architectural Interfaces

The `[[manifest_registry_spec]]` defines how this node is indexed by the `[[files_executive_agent]]`.

References

- `[[hierarchical_manifest_routing]]`
- `[[procedural_tool_extraction]]`

Tier 3: `03_recall_cache/` (Working Memory Accelerator)

- **Write Policy:** Agent-generated during build or indexing steps. May be deleted and rebuilt safely.
- **Chunk Schema** (`03_recall_cache/jsonl/*.jsonl`):

```
{"chunk_id": "CHK-8901", "parent_doc": "02_wiki_md/concepts/rlvr.md", "tokens": 256,
"content": "RLVR verifiers score execution traces against deterministic unit tests...",
"metadata": {"tier": 2, "topic": "rlvr"}}
```

Tier 4: `04_skills_runtime/` (Procedural Memory: Skills & Tools)

- **Write Policy:** Version-controlled tooling, prompts, and instructions.
- **Prompt Skill Format** (`prompt_skills/`): Standard modular prompt format containing purpose, required inputs, tool dependencies, and output schemas (`SKILL.md`).
- **Extracted Tools** (`extracted_tools/`): Standalone executable code blocks extracted from documentation. Non-interactive scripts with standard exit codes (`0` = success, non-zero = error).

Tier 5: `05_episodic_logs/` (Episodic Memory & Telemetry)

- **Write Policy:** Append-only during agent task execution.
- **Trajectory Schema** (`trajectories/YYYYMMDD_session.jsonl`):

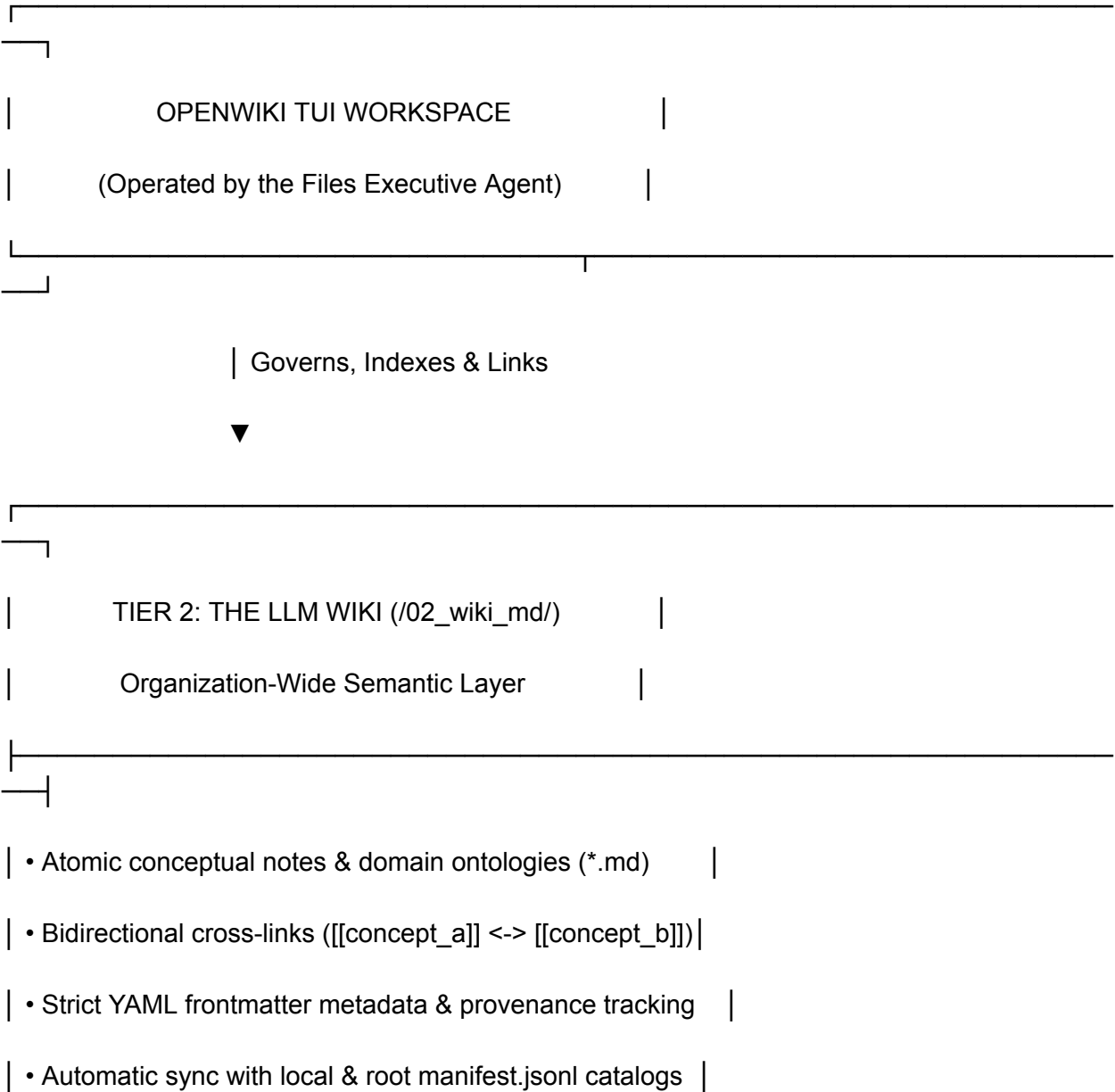
```
{"timestamp": "2026-09-03T17:01:35Z", "step": 1, "task_id": "TASK-104", "prompt_hash":
"a1b2c3", "tool_call": "run_linter", "exit_code": 0, "response_snippet": "OK"}
```

- **RLVR Log Schema** (`rlvr_verifiers/YYYYMMDD_eval.jsonl`):

{ "timestamp": "2026-09-03T17:01:40Z", "task_id": "TASK-104", "verifier_id": "syntax_test", "reward": 1.0, "feedback": "All assertions passed." }

4. The LLM Wiki Protocol & OpenWiki Files Executive Agent

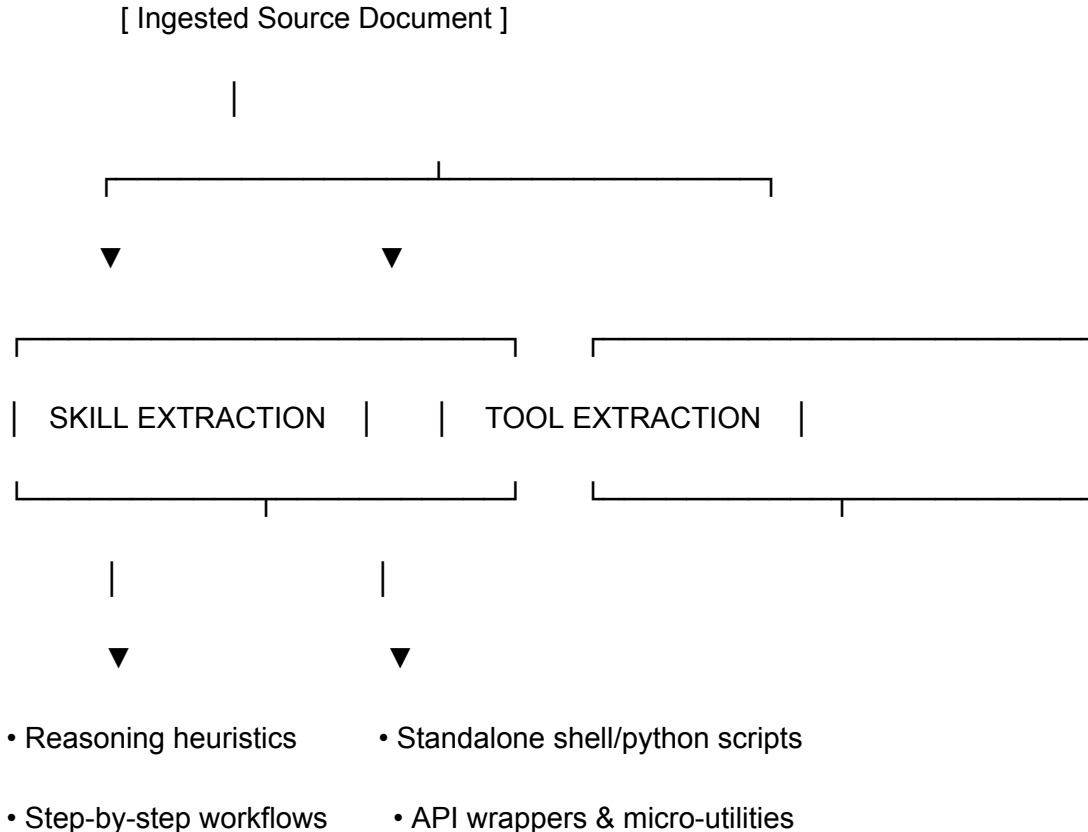
The LLM Wiki ([02_wiki_md/](#)) serves as the persistent organizational spine. The Files Executive Agent manages it through the OpenWiki TUI:



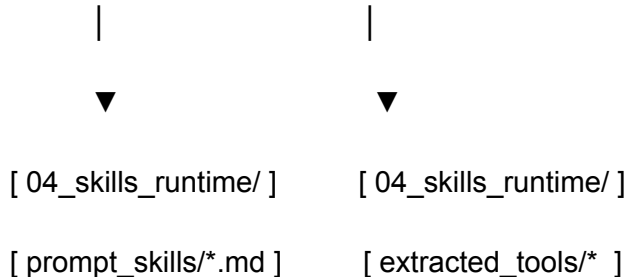
-
1. **TUI-Driven Lifecycle Management:** The Files Executive Agent runs inside the OpenWiki TUI, monitoring file additions, moves, renames, and refactors across all tiers.
 2. **Deterministic Linking & Backlinks:** Automatically verifies bidirectional Markdown links (`[[entity_name]]`), ensuring the LLM Wiki remains an intact graph without orphaned concepts.
 3. **Extraction Intake Coordination:** Acts as the bridge between incoming raw sources in `01_raw_sources/`, synthesizing structured knowledge into `02_wiki_md/`, and passing extracted procedural artifacts to `04_skills_runtime/`.
 4. **Manifest Synchronization:** Whenever the agent modifies a node in `wiki_md/`, it immediately writes updated hashes, token counts, and entity tags to both `02_wiki_md/manifest.jsonl` and the root `manifest.jsonl`.
-

5. Universal Skill & Tool Extraction Engine

The compilation agent processes incoming documentation through a dual-channel extraction filter:



- Formatting / prompt templates • CLI invocation syntax & flags
- Edge-case recovery rules • Data conversion routines



Extraction Classification Rules

1. **Targeting Skills:** If a passage describes how to reason, how to structure an analysis, or how to coordinate a sequence of tasks, format it as a markdown skill with YAML frontmatter into `04_skills_runtime/prompt_skills/`.
2. **Targeting Tools:** If a passage contains executable code, CLI command flags, API request formats, or data transformation utilities, extract it into an executable script (`.py` or `.sh`) with explicit argument parsing into `04_skills_runtime/extracted_tools/`.
3. **Registration:** Update the local `04_skills_runtime/manifest.jsonl` immediately upon extraction.

6. Hierarchical `manifest.jsonl` Specification for Distributed RAG

Every directory root must hold a dedicated `manifest.jsonl` file. This prevents full-vault scanning during localized RAG operations.

JSONL Schema Structure

Each line in any `manifest.jsonl` must strictly adhere to this schema:

```

{
  "id": "UUID-OR-PATH-HASH",
  "path": "relative/path/to/file.ext",
  "tier": 4,
  "category": "extracted_tool",

```

```
"sha256": "e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855",

"tokens": 412,

"semantic_summary": "CLI utility to extract audio streams from MP4 video containers.",

"entities_extracted": ["ffmpeg", "audio_processing", "mp4_to_wav"],

"skills_tools_extracted": [

  {"type": "tool", "name": "extract_audio_stream", "path":
"04_skills_runtime/extracted_tools/cli/extract_audio.sh"},

  {"type": "skill", "name": "audio_preprocessing_policy", "path":
"04_skills_runtime/prompt_skills/audio_prep.md"}

],

"provenance_source": "01_raw_sources/pdf/media_processing_guide.pdf",

"last_synced": "2026-09-03T17:05:00Z"

}
```

RAG Query Traversal Protocol

1. **Directory-Level Scope Matching:** The RAG agent inspects root `manifest.jsonl` to identify candidate subdirectories.
2. **Localized Search:** The agent reads only the relevant sub-tier `manifest.jsonl` (e.g., `04_skills_runtime/manifest.jsonl`), matching semantic summaries and entities.
3. **Targeted Ingestion:** Only the exact chunk or tool indicated by the manifest record is loaded into active Working Memory.

7. End-of-Day P2P Sync, Cross-Auditor & Sandboxed Red Auditor Workflow

When the daily driver reconnects to the home environment via the peer-to-peer server mesh, the Home Script Collector / Official Auditor executes the cross-audit:

| DAILY DRIVER (Edge Run) |

| - Query Model Traces |

| - Executor Model Logs |

| - Frontier/Cloud API Logs |

|

| P2P Sync (End of Day)



|

| HOME NODE: Official Cross-Auditor & Script Collector Agent |

|

| 1. Ingests raw session logs into: 05_episodic_logs/daily_driver_sync/ |

| 2. Reconstructs full inference trajectories across models: |

| [User Intent] —► [Query Model] —► [Executor Model] —► [Frontier/Cloud Fallback] |

| 3. Judges compiled tool usages and scripts against Universal Source of Truth (DB/Vault) |

|

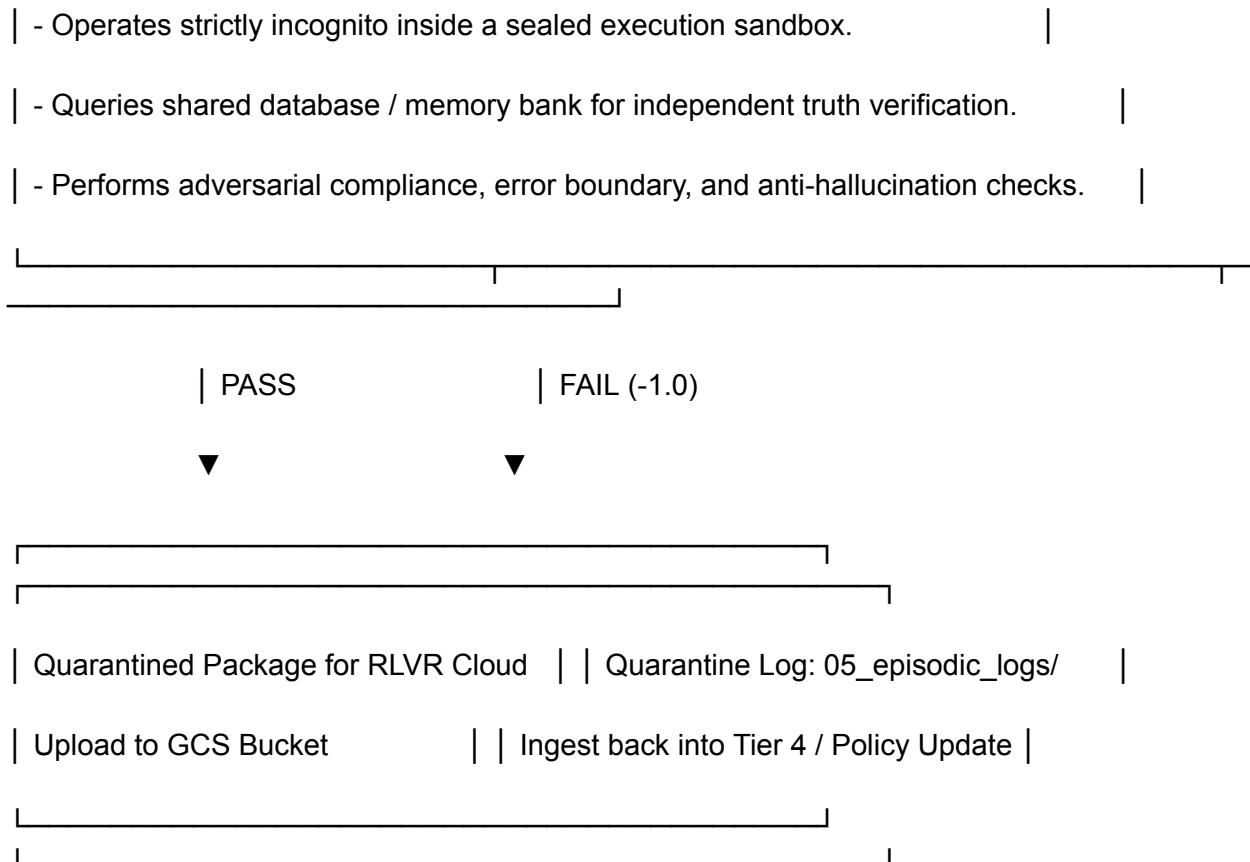
| Unverified Trajectories (Trap Door)



|

| HOME NODE: Sandboxed Red Auditor Gatekeeper (.incognito_red_sandbox/) |

|



Tri-Model Trace Alignment

1. **Query Model:** Captures initial intent decomposition, system prompts, and task planning.
2. **Executor Model:** Captures deterministic tool invocations, bash runs, and code snippets.
3. **Frontier Model:** Captures fallback escalations, complex conceptual synthesis, and recovery reasoning.

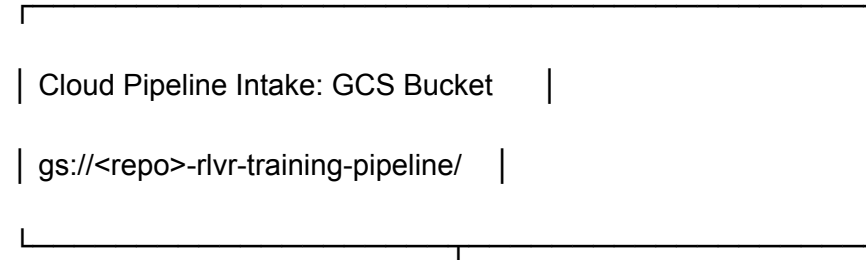
Sandbox Evaluation & Dispositions

- **Determinism:** The extracted script must execute in isolation without hidden dependencies.
- **Grounding Authority:** Must agree with the universal memory bank without invented parameters.
- **Dispositions:**
 - **VERDICT_APPROVED (+1.0):** Moved to upload queue for recursive training.
 - **VERDICT_REJECTED (-1.0):** Logged to `05_episodic_logs/.incognito_red_sandbox/` with an explicit failure reason, triggering recursive self-correction in Tier 4 policies.

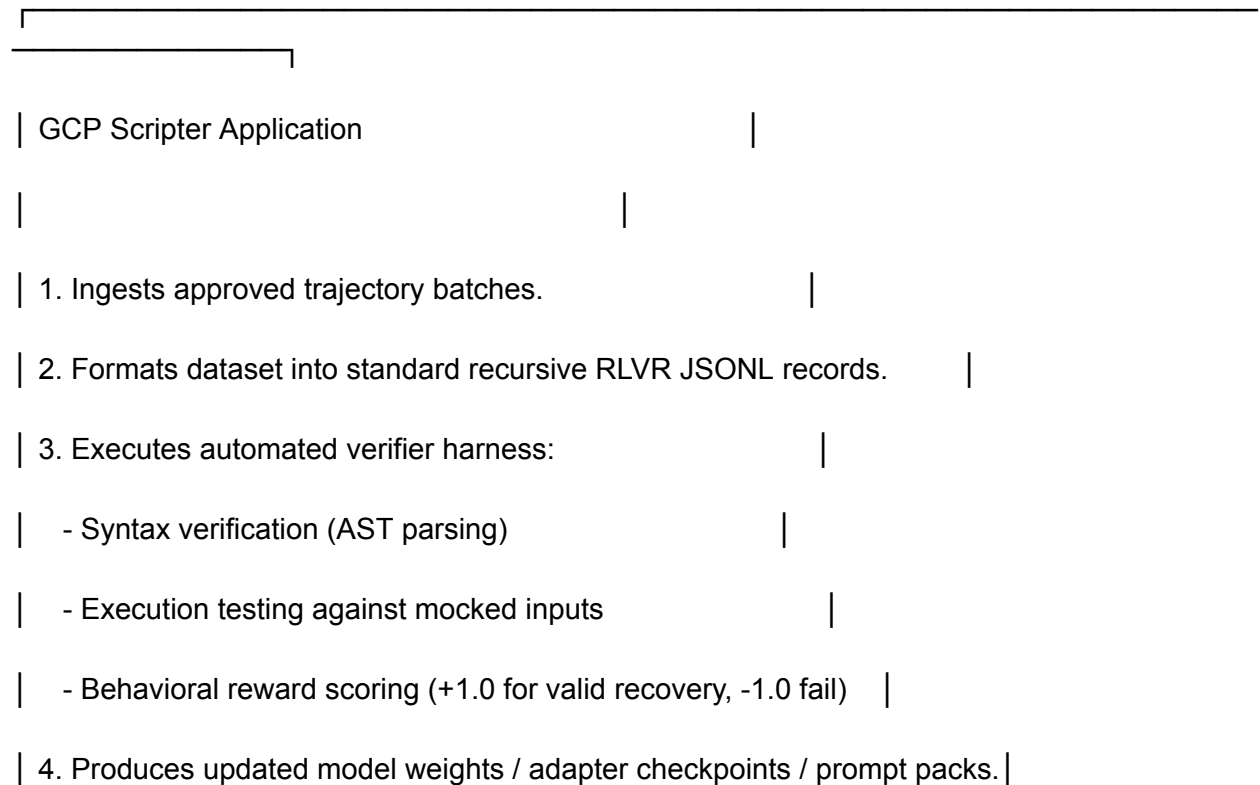
8. Cloud Scripter & RLVR Recursive Training Pipeline

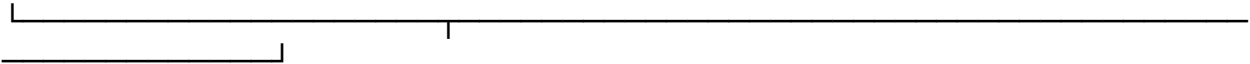
[Approved Trajectories]

|

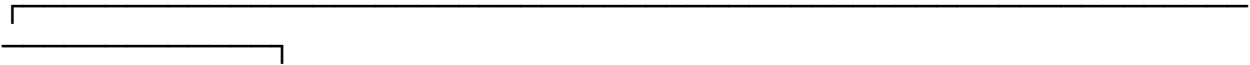


| Triggers Cloud Function / Cloud Run





| Downstream Artifact Sync

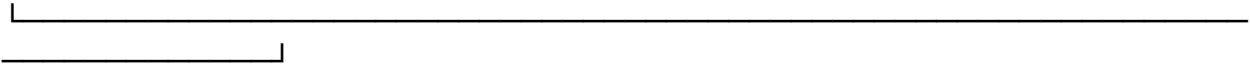


| Deployment Back to Edge & Local Tiers |

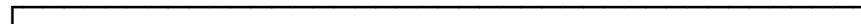
| - Updated Prompt Skills —▶ 04_skills_runtime/prompt_skills/ |

| - Updated Extracted Tools —▶ 04_skills_runtime/extracted_tools/ |

| - Verified Run Logs —▶ 05_episodic_logs/rlvr_verifiers/ |



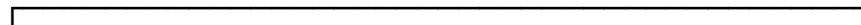
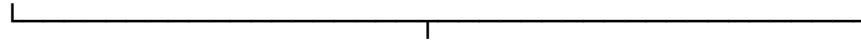
9. Agent Execution & Planning Protocols



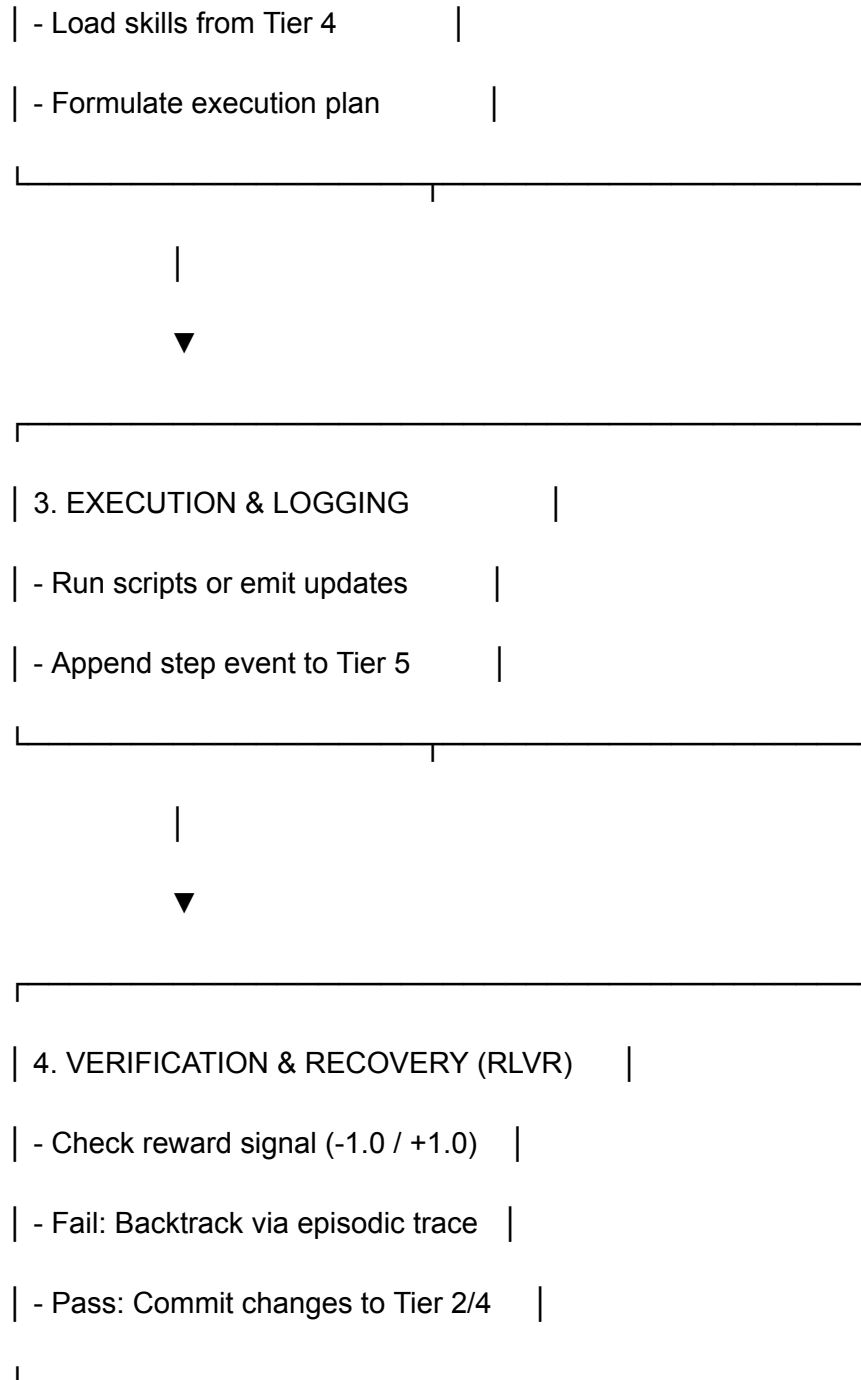
| 1. INGESTION & PLAN STAGE |

| - Consult MAP.md / manifest.jsonl |

| - Pull fast context from Tier 3 |



| 2. REASONING & PROCEDURAL DISPATCH |



Operational Steps for the Planning Agent

- **Step 1: Locating Information (Read Pass):**
 1. When asked a domain question, check [03_recall_cache/](#) first for relevant chunks.
 2. If full context is required, resolve the source path via [MAP.md](#) and read the relevant note in [02_wiki_md/](#).

3. Only query `01_raw_sources/` if source verification or citation check is explicitly requested.
 - **Step 2: Scaffolding or Modifying Repo Content (Write Pass):**
 1. Place original asset into `01_raw_sources/<type>/`.
 2. Synthesize conceptual notes into `02_wiki_md/<category>/` via OpenWiki TUI conventions.
 3. Extract procedural skills to `04_skills_runtime/prompt_skills/` and executable tools to `04_skills_runtime/extracted_tools/`.
 4. Regenerate search chunks in `03_recall_cache/jsonl/`, update sub-tier `manifest.jsonl`, and append record to root `manifest.jsonl` and `MAP.md`.
 - **Step 3: Executing Tools & Self-Correction (RLVR Loop):**
 1. Retrieve relevant executable scripts or skills from `04_skills_runtime/`.
 2. Stream execution status and tool payloads into `05_episodic_logs/trajectories/`.
 3. If a step fails (reward: -1.0 or non-zero exit code): ingest error trace from Tier 5 into active Working Memory, search `04_skills_runtime/policies/` for documented recovery heuristics, attempt remediation, and log retry.
-

10. Complete Repository Scaffolding Script

Execute this non-interactive bash sequence to bootstrap or audit the entire 5+1 Tier cognitive filesystem topology:

```
#!/usr/bin/env bash
```

```
set -euo pipefail
```

```
echo "===> Initializing 5+1 Tier Cognitive Repository Architecture..."
```

```
# 1. Scaffold all directory hierarchies
```

```
mkdir -p 01_raw_sources/{pdf,media,text}
```

```
mkdir -p 02_wiki_md/{concepts,architectures,entities,indexes}
```

```
mkdir -p 03_recall_cache/{jsonl,vectors,kv_store}
```

```
mkdir -p 04_skills_runtime/{prompt_skills,extracted_tools/{cli,wrappers},runtimes,policies}
```

```
mkdir -p
05_episodic_logs/{daily_driver_sync,trajectories,.incognito_red_sandbox,rlvr_verifiers,hygiene_r
eports}
```

2. Touch distributed manifest.jsonl files across all tiers

```
touch manifest.jsonl
```

```
touch 01_raw_sources/manifest.jsonl
```

```
touch 02_wiki_md/manifest.jsonl
```

```
touch 03_recall_cache/manifest.jsonl
```

```
touch 04_skills_runtime/manifest.jsonl
```

```
touch 05_episodic_logs/manifest.jsonl
```

3. Initialize Root MAP.md if absent

```
if [ ! -f "MAP.md" ]; then
```

```
    cat << 'EOF' > MAP.md
```

Master Repository Ontology Map

01. Raw Sources (Cold Archive)

- Sensory ground truth cataloged in `01\_raw\_sources/manifest.jsonl`.

02. The LLM Wiki Layer (`wiki\_md/`)

- Concepts: `02\_wiki\_md/concepts/`

- Architectures: `02\_wiki\_md/architectures/`

- Entities: `02\_wiki\_md/entities/`

- Indexes (MOCs): `02\_wiki\_md/indexes/`

- Governed via OpenWiki TUI by the Files Executive Agent.

03. Recall Cache (High-Speed Working Memory)

- Pre-tokenized Chunks: `03\_recall\_cache/jsonl/`

- Vector Indices: `03\_recall\_cache/vectors/`

- KV Store: `03\_recall\_cache/kv\_store/`

04. Procedural Runtimes, Skills & Extracted Tools

- Prompt Skills: `04\_skills\_runtime/prompt\_skills/`

- Extracted Tools: `04\_skills\_runtime/extracted\_tools/`

- Execution Runtimes: `04\_skills\_runtime/runtimes/`

- Policies & Guards: `04\_skills\_runtime/policies/`

05. Episodic Logs & Trajectories

- Daily Driver Sync: `05\_episodic\_logs/daily\_driver\_sync/`

- Trajectories: `05\_episodic\_logs/trajectories/`

- Red Audit Sandbox: `05\_episodic\_logs/.incognito\_red\_sandbox/`

- RLVR Verifiers: `05\_episodic\_logs/rlvr\_verifiers/`

- Hygiene Reports: `05\_episodic\_logs/hygiene\_reports/`

EOF

fi

echo "[✓] 5+1 Cognitive Repository Architecture Scaffolding Complete."